



The Open University

M263 Unit 3

UNDERGRADUATE ICT AND COMPUTING

Building Blocks of Software



Data types

Unit 3



The Open University

M263 Building Blocks of Software

Unit 3

Data types

Study guide

There is a software package, which we refer to as the **WorkPad**, for you to use. This package expresses the course code in a way that can be executed on a computer. You can find details of how to install the WorkPad in the *Guide to the Course*. It is intended that you spend part of your study time for this unit working with the WorkPad. How you wish to arrange your practical computing work is up to you. Depending on your personal circumstances and preferred method of studying, you may choose to:

1. do all the practical computing work when you have finished studying the unit text;
2. do practical computing work after completing your study of each of Sections 2, 3 and 4;
3. work on the computer in parallel with the study of the text (starting with Section 2).

None of these options is incompatible with the arrangement of the material.

We suggest that you should spend about one quarter of your study time for this unit on practical work on the computer, with the rest divided equally between study of Sections 1 and 2 (combined), Section 3 and Section 4.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2688 6

Contents

Introduction	4
1 The data types Bool, Int and Char	5
1.1 The data type Bool	6
1.2 The data type Int	7
1.3 The data type Char	11
1.4 Using data types in implementations	13
2 Writing code for the WorkPad	15
3 Arrays	23
4 Stacks	34
Summary of Unit 3	42
Solutions to the exercises	43
Index	48

Introduction

You saw in Unit 2 that the course code requires you to state what type of data is to be stored by any local variable used in code. This information provides an indication of the storage required for a variable. It does not, though, let you know what processes can be applied to the variable. In this unit, we shall codify the processes that are available when writing code. This is done through the idea of **data type**. From now on, both local variables and signatures of processes will normally be given in terms of data types.

A data type consists of a particular form of data together with processes that are available to manipulate data of that form. For example, a character data type might consist of the set of characters *Char*, together with the functions *ASC* and *CHR*. (These were all described in Unit 1.) However, it is usual to have other processes available, such as the comparisons $=_{Char}$ and $<_{Char}$.

In Section 1, three data types are described. Each is based on a form of data and functions that you met in Units 1 and 2. These data types are based on, respectively: characters, integers and Boolean values.

The processes associated with a data type are known as the **type functions** of that data type. Since the processes that are available will affect the way you can work with the data, data types are regarded as different if they have different type functions, even if the underlying form of data is the same.

In Sections 3 and 4 we will be concerned with two different data types that have the same underlying form of data. In each case, the form of data consists of a sequence of items. (The items in the sequence come from some arbitrary set, *X*.)

In Section 3, the data type **Array of X** is described. Variables of this type are called **arrays**. An array variable is a sequence containing a fixed number of items. The type functions on arrays allow you to examine the item at a specified position in an array, or to change the entry at a specified position in an array. However, you cannot change the size of the array.

In Section 4, we describe a data type **Stack of X**. This is another data type whose underlying form of data is a sequence. We refer to variables of this type as **stacks**. The number of items in a variable of stack type is not fixed. The type functions on a stack allow you to work at only one end of the sequence; so a stack is often pictured as a pile of items, whose bottom is not accessible. Items can be added or removed from the top of the “pile”, and the item at the top can be examined.

Characters, integers and Boolean values are fundamental forms of data. In this course, we shall not discuss how these data types might be represented on a machine. In later units, we shall discuss how we can build other data types, based on the assumption that certain **primitive data types** are available. The data types described in this unit are regarded as primitive.

Determining appropriate types to work with is a key aspect of the design of large software systems. However, we shall not be concerned here with designing or choosing types for an application. We are concerned with principles underlying the practice. These principles can be obscured by complicated detail. Hence we shall deal with a fairly small range of types, and these will usually make available a rather limited range of processes. This would not necessarily be convenient in practice, but serves our purposes better in the course.

There is a software package for you to use that expresses the course code in a way that can be executed on a computer. This package is referred to as the **WorkPad**. To enable code to be entered on a computer, we need to express its statements using only symbols available on a standard keyboard. For example, the symbol $\leftarrow$ used in Unit 2 to express assignment is not available on a keyboard. For the WorkPad, we need to write code with $\leftarrow$ replaced by the three key strokes `<--`. We need to make a number of minor modifications to the way we express the course code to make it correspond directly to the form that you will key in for the WorkPad. These modifications are brought together in Section 2.

1 The data types **Bool**, **Int** and **Char**

A **data type** consists of a form of data together with certain processes associated with data of the specified form. These processes are called the **type functions** of the data type. In this section we will describe three data types, based on Booleans, integers and characters. To a certain extent this section is about organising material that you have already met, since these forms of data and most of the associated type functions were mentioned in Units 1 and 2. However, there are some new ideas here. We will be able to consolidate ideas about type consistency in code that were mentioned in Unit 2. We shall also introduce the concept of **overloading**, which is the use of the same identifier for different processes. For example, we shall use the same infix notation (`==`) for tests of equality on both characters and integers, rather than distinguishing them by adding subscripts, as we did in Unit 2 when we wrote `=Char` and `=Int`.

The first data type that we will describe has, as its form of data, the Boolean values **true** and **false**. Its type functions are *NOT*, $\wedge$ (and), $\vee$ (or), and an equality test. The name of this data type is **Bool**. Before giving a full description of the data type **Bool**, there are a few points to make that apply to all the primitive data types.

Type functions may include binary operations written using infix notation.

In Unit 2, we gave variables in code a **type** by stating a set. From now on, we shall state their data type. Similarly, we shall use data types when giving the signature line in a specification. Declaring the type of a variable tells us two things. The first is the form of data that is stored. The second is the processes that are available to that variable in code.

Underlying each data type is a set, which we shall call the **data set**. This set gives the form of data. For example, in the data type **Bool**, the data set is $Bool = \{\mathbf{true}, \mathbf{false}\}$. The data set gives the set of possible states that may be taken by a variable of that data type. Thus a variable of type **Bool** has two possible states: **true** or **false**. The data type carries more information than the data set, since it tells you not only the form of data, but also the processes that are available. Note that, although we often use the same identifier for a data type and its underlying data set, we distinguish between them by using bold print for the data type and italics for the data set.

You will not be expected to use bold or italic type (or to attempt handwritten equivalents) in your own work.

A third point concerns the way we identify the type functions. Each type function has an **identifier**, used to call the process in code. We shall give all type functions identifiers that can be entered at a computer keyboard.

For example, we use the single symbol $\wedge$ to identify the binary operation “and” on Booleans, but this symbol is not available on a keyboard. The identifier of the corresponding type function of **Bool** consists of two key strokes: `/\` (slash then backslash). Similarly, the type function “or” ($\vee$) has an identifier comprising the two keystrokes `\/` (backslash then slash).

In such cases, we will expect you to be able to interpret both notations. The single symbol may be used in text, but in code we will use the identifier of the type function, as entered from a keyboard.

The data type **Bool** has a type function of equality, which compares two Boolean values and returns **true** if these are the same and **false** if they are not. This function has the infix identifier `==`. Use of two equals signs is common practice in programming languages, and enables one to distinguish between a binary operation available in code to test whether two values are the same, and the many other uses to which a single equals sign may be put in computing and mathematics.

1.1 The data type **Bool**

The data type **Bool** is summarised in the box below. This summary first gives the form of data, which is a value from $Bool = \{\mathbf{true}, \mathbf{false}\}$, the data set. It then gives the type functions and their signatures. In this case these are the function *NOT*, the binary operations $\wedge$ and $\vee$, and one comparison, equality.

Bool	Data type
<i>Form of data</i> Values from <i>Bool</i>, that is, true or false. <i>Type functions</i> function <i>NOT</i>(<i>x in Bool</i>) return in Bool function (infix) (<i>x</i> $\wedge$ <i>y on Bool</i>) return in Bool ($\wedge$ is entered on the keyboard as <code>/\</code>.) function (infix) (<i>x</i> $\vee$ <i>y on Bool</i>) return in Bool ($\vee$ is entered on the keyboard as <code>\/</code>.) function (infix) (<i>x</i> <code>==</code> <i>y on Bool</i>) return in Bool	

Here and in the other class definitions we have used the standard notation for identifiers. Where necessary we have indicated how these are entered on the keyboard when using the WorkPad.

To keep the boxed summary of the data type short, we give only the signature lines of the type functions. To give a complete description of the data type, we would need to give a full specification, including the precondition and postcondition, for each type function. Specifications of *NOT*, $\wedge$ and $\vee$ were given in Unit 1 and are not repeated here. The specification of the equality test is given below.

function (infix) (*x* `==` *y on Bool*) **return in Bool**
pre **true**.
post The returned value is **true** if *x* and *y* are the same, and is **false** otherwise.

The descriptions of data types given in the *Course Handbook* include full specifications of each type function for reference.

Activity 1.1

Suppose that $a = \mathbf{true}$, $b = \mathbf{true}$ and $c = \mathbf{false}$. Give the values of each of the following expressions.

- (a) $a \vee b$.
- (b) $\mathbf{NOT}(b \wedge c)$.
- (c) $a \wedge (b \vee c)$.
- (d) $(a \wedge b) \vee (a \wedge c)$.

Discussion

- (a) This is $\mathbf{true} \vee \mathbf{true} = \mathbf{true}$.
- (b) First evaluate the part of the expression in brackets:
 $b \wedge c$ is $\mathbf{true} \wedge \mathbf{false} = \mathbf{false}$.
 So $\mathbf{NOT}(b \wedge c) = \mathbf{NOT}(\mathbf{false}) = \mathbf{true}$.
- (c) We have $b \vee c = \mathbf{true} \vee \mathbf{false} = \mathbf{true}$.
 So $a \wedge (b \vee c) = \mathbf{true} \wedge (\mathbf{true} \vee \mathbf{false}) = \mathbf{true} \wedge \mathbf{true} = \mathbf{true}$.
- (d) We have

$$(a \wedge b) \vee (a \wedge c) = (\mathbf{true} \wedge \mathbf{true}) \vee (\mathbf{true} \wedge \mathbf{false})$$

$$= \mathbf{true} \vee \mathbf{false} = \mathbf{true}.$$

As noted in Unit 2, in many programming languages, a statement equivalent to

var *aBool* **in** **Bool**

would create the variable *aBool* with a particular stored value (the default value). However, the data types described in this section do not have default values. This means that a variable of type **Bool** (or **Int** or **Char**) is created without any stored value, and must be assigned an initial value before you can refer to it in other statements.

1.2 The data type Int

The integers are whole numbers, including positive numbers such as 7 or 11546, the number 0, and negative numbers such as -436 . The data type **Int** has data set *Int*, which you met in Unit 1. Many of the type functions of **Int** were also mentioned in Unit 1. The binary operations of addition, subtraction and multiplication are familiar from ordinary arithmetic. The comparisons $<$, $>$ and equality are also familiar. Recall that each comparison returns a Boolean value. One point to note is that any negative number is smaller than any positive number: for example, $-123 < 5$ is **true**. Since the symbol $\times$ is not on the keyboard, it is necessary to use a different identifier for multiplication in code: use of $*$ for multiplication is common practice in programming languages. (In the text, we shall continue to use $\times$ for multiplication.)

The data type **Int** has a type function “less than or equal to”. In mathematics, this condition is written as $x \leq y$. Although the condition $x \leq y$ is identical to $(x < y) \vee (x = y)$, it is convenient to have this as a separate type function to reduce the size of expressions appearing in code. We shall use the notation $\leq$ in text, but the identifier `<=` must be used in code.

The symbol $\leq$ is not on the keyboard.

The binary operations of integer division (*DIV*) and the associated remainder (*MOD*) were discussed in Unit 1. Some examples are given below.

$$\begin{aligned} 30 &= 6 \times 5 + 0 & \text{so } DIV(30, 5) &= 6; & MOD(30, 5) &= 0. \\ 11 &= 2 \times 4 + 3 & \text{so } DIV(11, 4) &= 2; & MOD(11, 4) &= 3. \\ -11 &= (-3) \times 4 + 1 & \text{so } DIV(-11, 4) &= -3; & MOD(-11, 4) &= 1. \end{aligned}$$

For any integers x and y , we have $x = y \times DIV(x, y) + MOD(x, y)$, where the remainder $MOD(x, y)$ satisfies the condition $0 \leq MOD(x, y) < y$. Note that *DIV* and *MOD* are written using function (rather than infix) notation.

We use the identifier `==` for the equality test for integers. This is the same identifier as we use for equality of Boolean values (and, as you will see, it is also used for equality of characters). To use the same identifier for equality tests for integers, for Booleans and for characters is normal practice in programming languages. However, it means that the same identifier is being used for different processes. Integers and characters will be represented in different ways on a machine. When `==` is called, different routines will be needed to implement the test of equality, depending upon whether characters, integers or Booleans are being compared.

Unlike in Units 1 and 2, we have not attached subscripts to the identifiers of any of the type functions on integers, and you will see shortly that we have not done so for type functions on characters, either.

Int	Data type
<i>Form of data</i>	
Whole numbers: positive, negative or 0.	
<i>Type functions</i>	
function (infix) $(x + y \text{ on Int})$ return in Int	
function (infix) $(x \times y \text{ on Int})$ return in Int	
(* is used for $\times$ in the WorkPad.)	
function (infix) $(x - y \text{ on Int})$ return in Int	
function (infix) $(x == y \text{ on Int})$ return in Bool	
function (infix) $(x < y \text{ on Int})$ return in Bool	
function (infix) $(x > y \text{ on Int})$ return in Bool	
function (infix) $(x \leq y \text{ on Int})$ return in Bool	
($\leq$ is entered on the keyboard as <code><=</code> .)	
function (infix) $(x \geq y \text{ on Int})$ return in Bool	
($\geq$ is entered on the keyboard as <code>>=</code> .)	
function $MOD(x \text{ in Int}, y \text{ in Int})$ return in Int	
function $DIV(x \text{ in Int}, y \text{ in Int})$ return in Int	

We have not included “not equal to” as a separate type function of **Int**. The condition $x \neq y$ can be expressed as $NOT(x == y)$.

Full specifications were given for most of these type functions in Unit 2 and are not repeated here. Remember that, for the data type **Int**, `==` identifies the binary operation specified as $=_{Int}$ in Unit 2. For completeness, we give

below specifications of the two binary operations that we did not discuss in Unit 2: $\leq$ and $\geq$.

function (infix) ($x \leq y$ on Int) return in Bool

pre true.

post The returned value is **true** if the integer x is less than or equal to the integer y and is **false** otherwise.

function (infix) ($x \geq y$ on Int) return in Bool

pre true.

post The returned value is **true** if the integer x is greater than or equal to the integer y and is **false** otherwise.

We recommend use of the *Course Handbook* for reference when you want to remind yourself of the details of any data type. Full specifications of all the type functions are given therein.

Activity 1.2

- (a) Give the values of:
 - (i) $MOD(23, 10) \leq MOD(33, 10)$;
 - (ii) $MOD(23, 10) < MOD(33, 10)$.
- (b) With $x = 23, y = 4$ and $z = 5$, give the value of each of the following:
 - (i) $MOD(x, y)$;
 - (ii) $DIV(x, y)$;
 - (iii) $(MOD(x, y) == MOD(x, z)) \wedge NOT(z == y)$.

Discussion

- (a) $MOD(23, 10) = 3$ and $MOD(33, 10) = 3$. So the expression in (i) becomes $3 \leq 3$, which is **true**, while the expression in (ii) becomes $3 < 3$, which is **false**.
- (b) We have $23 = 5 \times 4 + 3$; so the remainder when 23 is divided by 4 is 3, and the result of integer division of 23 by 4 is 5. So:
 - (i) $MOD(23, 4) = 3$;
 - (ii) $DIV(23, 4) = 5$.
 - (iii) When we divide 23 by 5, we get $23 = 4 \times 5 + 3$, and so the remainder is 3. So $MOD(23, 4) == MOD(23, 5)$ is **true**. Now $z == y$ gives $5 == 4$, which is **false**, and so $NOT(z == y)$ is **true**. Thus the given expression is **true** $\wedge$ **true** = **true**.

In practice, **Int** will need an additional feature, which is not reflected in the description of this data type given above. A computer will not be able to store integers of arbitrary size. There will be a bound on the size of integer that can be stored, such as 2^{32} . So in a programming language, an integer data type will include only integers x satisfying a condition such as $-2^{32} < x < 2^{32}$. Several of the type functions will then need a precondition stating that the returned value lies in this range. As the bound on the size of an integer that can be stored will be very large, and will not affect your work in this course, we have not included such a condition in our description of **Int**.

Now even if one starts with whole numbers, processes such as division or finding a square root often yield results that are not integers. We refer to the wider collection of numbers, including numbers such as $\frac{122}{7}$, $\sqrt{2}$ and $\pi (= 3.14159 \dots)$ as **real numbers**. A real number is conveniently thought of as a decimal. For example, $\frac{122}{7}$ might be expressed in decimal notation in a form such as $17.4285714 \dots$. Most programming languages will include a data type for handling numbers that are not integers, usually called **Float**. However, we will not describe a data type for handling numbers other than integers. We touched on the reasons for this in Unit 1, but will reprise them here.

This use of the term “real number” here is that used in mathematics. Not all programming languages use it in this way.

Real numbers in general can only be stored with a limited accuracy. They are likely to be represented in **floating point** form. For example, if we were restricted to 6 decimal places, the number 17.4285714 would be expressed as 0.174286×10^2 . Now, for numbers in floating point form, the standard arithmetic operations cannot be performed with complete accuracy. Suppose that x and y are numbers expressed in floating point form to an accuracy of 6 decimal places, and that we specify an operation $x +_{\text{FLOAT}} y$ that returns a value that is also a number expressed to an accuracy of 6 decimal places. Then the value returned for $x +_{\text{FLOAT}} y$ will only be an approximation to the exact value of $x + y$. For example:

$$0.815233 \times 10 + 0.174286 \times 10^2 = 0.2558093 \times 10^2,$$

which when restricted to 6 decimal places is 0.255809 . In this case, there is only a slight inaccuracy in the returned value, of course. However, in longer calculations, these small inaccuracies can be propagated in complicated ways. For example, suppose that x and y are numbers expressed with small inaccuracies, and that x is close to y but is not the same. Then we may find that $x == y$ returns the value **true** when we expect **false**, and we may find that if we calculate $\frac{1}{(x-y)}$ then the level of inaccuracy is suddenly very great. Calculations with floating point numbers can often be carried out without difficulty. However, it is not easy to specify operations on floating point numbers with the sort of precision that we wish to deal with in this course. To do that would raise complications that are peculiar to the inaccurate arithmetic of floating point numbers, and not of more general interest.

It is often possible to use **Int** rather than **Float** when modelling real-world situations. For example, in Unit 1 we measured weights as a whole number of grams rather than a fractional number of kilograms, and rather than use a fractional number of pounds sterling, we used a whole number of pence.

1.3 The data type **Char**

As discussed in Unit 1, in this course we consider only characters in the set *Char*, with ASCII codes between 32 and 126 inclusive. These form the data set of the data type **Char**. Its type functions are the functions *ASC* and *CHR*, and the comparisons $=$, $<$, $>$, $\leq$ and $\geq$. Specifications of most of these functions were given in Unit 2. (*CHR* applied to an integer n requires the precondition $32 \leq n \leq 126$.)

Char	Data type
<i>Form of data</i> Characters from the set <i>Char</i>. <i>Type functions</i> function <i>ASC</i>(c in Char) return in Int function <i>CHR</i>(n in Int) return in Char function (infix) ($c == d$ on Char) return in Bool function (infix) ($c < d$ on Char) return in Bool function (infix) ($c > d$ on Char) return in Bool function (infix) ($c \leq d$ on Char) return in Bool ($\leq$ is entered on the keyboard as $<=$.) function (infix) ($c \geq d$ on Char) return in Bool ($\geq$ is entered on the keyboard as $>=$.)	

Activity 1.3

Give specifications for the type functions $\leq$ and $\geq$ of the data type **Char**.

Discussion The signature lines must show that the inputs to these type functions of **Char** are members of the set *Char*. For the semantics, remember that these comparisons are achieved by comparing the ASCII values of the input characters.

function (infix) ($c \leq d$ on **Char**) **return in Bool**

pre true.

post The returned value is **true** if $ASC(c) \leq ASC(d)$ and is **false** otherwise.

function (infix) ($c \geq d$ on **Char**) **return in Bool**

pre true.

post The returned value is **true** if $ASC(c) \geq ASC(d)$ and is **false** otherwise.

In Units 1 and 2 we were careful to distinguish between comparisons of integers and characters, and used subscripts to do that, as in $<_{Int}$ and $<_{Char}$ for example. As noted earlier, we have not attached subscripts to the identifiers of the type functions in **Int** and **Char**. For example, we have used $==$ for equality both of characters and of integers. The use of the same identifier for different processes is referred to as **overloading**. It is possible to interpret code involving overloaded identifiers. This is done by looking at the context in which the process identifier is used. Suppose, for example,

that x and y are both variables of type **Int**, while c and d are variables of type **Char**. Then if we write $c == d$ in code, this requires use of the binary operation $=_{Char}$, while if we write $x == y$, the binary operation $=_{Int}$ is required. The type of the variables being compared by $==$ tells us which process ($=_{Int}$ or $=_{Char}$) is required, and so it is not necessary in code to use different identifiers for these processes.

Now if you were to write $x == c$, attempting to compare an integer and a character, it would not be possible to apply either of the type functions ($=_{Int}$ or $=_{Char}$) identified by $==$. The type function of **Int** must be applied to two integers, while the type function of **Char** must be applied to two characters. This can be seen from the signature line of each function, given in its specification. Neither data type (**Char** or **Int**) has a type function $==$ that compares a character and an integer. Consequently, $x == c$ would be invalid course code. Note that this expression does not return **false**. It is an invalid expression.

This may sound rather complicated. However, it is not difficult to apply. In any piece of course code, all the variables will have a data type. If the code involves a process identifier, check whether there is a process with that identifier, and with the signature needed to make the code valid. If there is, and any preconditions are satisfied, then the code is valid. If there is not, then the code is not valid. The only point to be careful about is that different processes may share the same identifier.

Activity 1.4

Give the value of each of the expressions below, where it is valid.

- (a) $ASC('7') - 48$;
- (b) $ASC('7' - 48)$;
- (c) $NOT(('c' == 'd') \vee (ASC('c') < ASC('d')))$;
- (d) $(NOT('c' == 'd')) \vee (ASC('c') < ASC('d'))$.

Discussion

- (a) $ASC('7') = 55$, and then $ASC('7') - 48 = 55 - 48 = 7$. Note that this is now the integer 7, not the character '7'.
- (b) '7' is a character. The binary operation $-$ requires two integers as inputs. So the expression $'7' - 48$ is not valid (since the first input to $-$ is of the wrong data type), and so the given expression is not valid.
- (c) $'c' == 'd'$ is **false**, while $ASC('c') < ASC('d')$ is $99 < 100$, which is **true**. Then

$$\begin{aligned} NOT(('c' == 'd') \vee (ASC('c') < ASC('d'))) &= NOT(\mathbf{false} \vee \mathbf{true}) \\ &= NOT(\mathbf{true}) = \mathbf{false}. \end{aligned}$$

- (d) This expression is similar to that in (c), but the bracketing is subtly different. This is important! We have

$$\begin{aligned} (NOT('c' == 'd')) \vee (ASC('c') < ASC('d')) &= NOT(\mathbf{false}) \vee \mathbf{true} \\ &= \mathbf{true} \vee \mathbf{true} = \mathbf{true}. \end{aligned}$$

1.4 Using data types in implementations

From now on, we shall use data types when giving signature lines in specifications, and in **var** declarations in code. Doing this lets you know what processes can be applied to variables in valid code. For example, we give below a specification and implementation of a process to transform a character that is a number digit into the corresponding integer.

function *MAKEINT*(*c* in **Char**) **return** in **Int**

pre The character *c* has ASCII code between 48 and 57 inclusive.

post The returned value is the number corresponding to the character *c*.

```
/* impmakeint */
{
  var anInt in Int
  anInt ← ASC(c)
  anInt ← anInt − 48
  return anInt
}
```

We can check that this code is type consistent as follows. Look first at the assignment $anInt \leftarrow ASC(c)$. The function *ASC* requires an input of type **Char**. (We know this from the signature of *ASC*.) The signature line of *MAKEINT* shows that *c* is of type **Char**. The value returned by *ASC* is of type **Int** (from the signature of *ASC*). From the **var** declaration, we know that *anInt* is of type **Int**. So assigning *ASC*(*c*) to *anInt* is consistent with the declared type of the variable *anInt*.

Now look at the assignment $anInt \leftarrow anInt - 48$. The only type function with identifier $-$ is subtraction of variables of type **Int**. We know that *anInt* is of type **Int**, and we can recognise 48 as being a literal of type **Int**. So use of $-$ in $anInt - 48$ is consistent with the signature of the binary operation $-$. The value returned by $-$ is of type **Int**, and so assigning $anInt - 48$ to *anInt* is consistent with the declared type of the variable *anInt*. Finally, the value *anInt* returned by *impmakeint* is of type **Int**, as required by the signature of *MAKEINT*.

Activity 1.5

A specification taken from Unit 2 is given below. How should it be modified to take account of developments in the course code described in this section?

function *MAKEUPPER*(*c* in **Char**) **return** in **Char**

pre The character *c* is a lower-case letter.

post The returned value is the letter *c* in upper case. So, for example, if *c* = ‘q’ then the returned value is ‘Q’.

Discussion We need to modify the signature line to use data types, as below.

function *MAKEUPPER*(*c* in **Char**) **return** in **Char**

The course code is a **strongly typed** language. This means that the language has certain features relating to the typing of variables. (These features are not shared by all programming languages.) Features relating to this “strong typing” are listed below.

Type checking: a summary

1. Every variable has a type.
2. Every process has a signature.
3. Process application must be consistent with the signature of the process.
4. Assignment must be consistent with type. For primitive types, we can write $x \leftarrow y$ only if x and y have the same type.
5. Processes may share an identifier. We can use the context in which the identifier is used to determine which of the processes that share the identifier is intended.

We assume that, in code, the computer is able to make sense of an integer written as 48, for example. Indeed, for each of the data types **Int**, **Char** and **Bool**, we have a notation for writing particular constant values, which we assume can be interpreted by the computer. We refer to a constant value written in a notation that the computer can interpret as a **literal**. Where we have a notation for literals, this can be used in the course code. So far, you have met the following notation for literals:

for **Int**: decimal notation for integers, such as `-8`;
for **Char**: single inverted commas, as in `'A'`;
for **Bool**: **true** and **false**.

The terminology **decimal notation** is discussed in Section 2, if you have not seen it before.

We noted in Unit 2 that certain words in the course code, such as **for**, are known as reserved words and have special meanings. These reserved words should not be used as identifiers for variables used in code. For similar reasons, the names of data types should not be used as identifiers for variables. (This is why we use variable identifiers such as *aChar* and *anInt* rather than *char* and *int*.) Another point is that it is not advisable to use identifiers that differ only in their use of upper- and lower-case letters, as in *aChar* and *achar*, for example. Many programming languages would not distinguish between these. (The WorkPad *does* treat upper- and lower-case letters as different in variable identifiers.)

We use the term **case sensitive** to describe programming languages that distinguish between upper- and lower-case letters in identifiers.

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: data type; type function; identifier (of a process); overloading (of a process identifier).
- ☐ Understand that the functions available to a variable in code are determined by the variable's data type.
- ☐ Interpret and use the notation associated with the data types **Int**, **Bool** and **Char**, including that for their type functions and literals.
- ☐ Interpret expressions involving the type functions of the data types **Int**, **Bool** and **Char**.
- ☐ Check type consistency of statements involving the data types **Int**, **Bool** and **Char**.
- ☐ Check that assignments and function calls are type consistent when overloaded identifiers are used.
- ☐ Be aware that data type names should not be used as variable identifiers.

Exercises on Section 1

Exercise 1.1

Give the value of each of the following expressions, where they are valid.

- (a) $CHR(70) \geq 'b'$. (b) $DIV(DIV(37,5),3)$.

Exercise 1.2

- (a) If the variable c is of type **Char**, explain why the expression for $anexp$ below is type consistent, and give the type of $anexp$.

$$anexp = ((ASC(c) < 100) \vee (ASC(c) == 100)) \wedge (c > 'a').$$

- (b) For what values of c does the expression $anexp$ in (a) have the value **true**?
- (c) Give an expression (using type functions of the primitive data types) that is **true** for the same values of c as that in (a), and is briefer.

Exercise 1.3

Write the condition:

the character c is not an upper-case letter

using type functions of the primitive data types.

2 Writing code for the WorkPad

As noted earlier, when code is written in the course WorkPad, it needs to be expressed using only symbols available on a computer keyboard. The data types described in Section 1 use identifiers that take account of this. We also need to write `<--` rather than the single symbol `←` for assignment. When entering an implementation into the WorkPad, one needs to add an abbreviated version of the signature line from the specification as a first line.

For example, a specification and implementation taken from Unit 2 are given below.

function *MAKEUPPER*(*c* in *Char*) **return** in *Char*

pre The character *c* is a lower-case letter.

post The returned value is the letter *c* in upper case. So, for example, if *c* = 'q' then the returned value is 'Q'.

```
{
var num in Int
var aChar in Char
aChar ← c
num ← ASC(aChar)
num ← num - 32
aChar ← CHR(num)
return aChar
}
```

For the WorkPad, the implementation should be written as follows. (The first line is required as a signal that the code implements a function called *MAKEUPPER*, with a single input identified as *c*.)

```
function MAKEUPPER(c)
{
var num in Int
var aChar in Char
aChar <-- c
num <-- ASC(aChar)
num <-- num - 32
aChar <-- CHR(num)
return aChar
}
```

The WorkPad does not use formatting such as bold and italic. The background shading will be used in the text to pick out code written in a form suitable for the WorkPad. The shading is not part of the code!

The specification is not needed as part of the working code in the WorkPad. You may wish to include the specification (or a summary of it) as a comment, but that is optional. If a name for the implementation is wanted, that can also be included as a comment.

Incidentally, in an implementation you can, if you wish, return an expression rather than a single variable. For example, we could implement *MAKEUPPER* as follows.

```
function MAKEUPPER(c)
{
return CHR(ASC(c) - 32)
}
```

Such an approach does not show explicitly the steps in arriving at the final expression, which are:

1. Find the ASCII code of the input character.
2. Subtract 32 from that.
3. Find the character corresponding to this new integer.

To make the derivation of the code in our examples more clear, we have used a step-by-step approach. We have avoided the use of expressions that really express more than one step. The approach above is fine, but we would advise you to avoid using expressions that are excessively complicated since they can make code hard to follow and increase the chance of making errors in **syntax**.

Activity 2.1

An implementation that was given in Unit 2 is repeated below. Modify this to put it in a form suitable for the WorkPad.

function *ISCAP*(*c* in *Char*) **return** in *Bool*

pre **true**.

post The returned value is **true** if the character *c* is an upper-case letter and is **false** otherwise.

```
{
  var b in Bool
  b ← true
  if (ASC(c) <Int 65) then {b ← false}
  if (90 <Int ASC(c)) then {b ← false}
  return b
}
```

The term **syntax** refers to the rules of how valid expressions are constructed. For example, the placing of the brackets is part of the syntax. If $CHR(ASC(c) - 32)$ is changed to $CHR(ASC(c)) - 32$ by moving one bracket, then the expression is no longer defined — there is an error in the syntax.

Discussion The typing of the variable *b* must be expressed in terms of a data type, but this leads to no visible change in code. We need to write assignment as $\leftarrow$, to remove the subscripts from $<$, and to add a heading to the code in the implementation. The implementation with these changes is given below.

```
function ISCAP(c)
{
  var b in Bool
  b <-- true
  if (ASC(c) < 65) then {b <-- false}
  if (90 < ASC(c)) then {b <-- false}
  return b
}
```

EXAMPLE 2.1

The function *ABS* inputs an integer. If the input integer is negative, then the returned value “forgets” the minus sign. If the input integer is positive, then the returned value is the same as the input value. For example, $ABS(5) = 5$; $ABS(-7) = 7$; $ABS(0) = 0$. A specification and implementation of *ABS* are given below.

In mathematics, the notation $|n|$ is often used to mean $ABS(n)$ and is called the absolute value of n .

function *ABS*(*n* in **Int**) **return** in **Int**

pre **true**.

post The returned value is n if $n \geq 0$, and is $-n$ if $n < 0$.

```
function ABS(n)
{
  var anInt in Int
  anInt <-- n
  if (anInt < 0) then {anInt <-- (-1) * anInt}
  return anInt
}
```

In text, we shall continue to use formatting to pick out reserved words, data types and variables. But from now on, we will give implementations in the form in which they will appear in the WorkPad, with no formatting. This lack of formatting means that a statement in code such as

```
var anInt in Int
```

in Example 2.1 does not show explicitly that **Int** is a data type. From now on, all **var** statements in code should be interpreted as referring to a data type.

Activity 2.2

The function *UNIT* inputs a non-negative integer and returns the character that forms the last (rightmost) digit. So, for example, if the input is 3472 then the value returned is ‘2’. Give a specification and implementation of *UNIT*.

Discussion The remainder when the integer is divided by 10 gives the number we want. The code in the implementation must then turn this number into the appropriate character. A specification and implementation are given below.

function *UNIT*(*number* in **Int**) **return** in **Char**

pre $number \geq 0$.

post The returned value is the character giving the units digit in *number*. For example, if *number* is 3472 then the value returned is ‘2’. If *number* is 0, then the returned value is ‘0’.

```

function UNIT(number)
{
  var anInt in Int
  var aChar in Char
  anInt <-- MOD(number,10)
  /* anInt is now the desired number. */
  anInt <-- anInt + 48
  /* anInt is now the ASCII code of
  the desired number digit. */
  aChar <-- CHR(anInt)
  /* aChar is the required character. */
  return aChar
}

```

The following is an alternative implementation.

```

function UNIT(number)
{
  return CHR(MOD(number,10) + 48)
  /* The last digit as a number is obtained by finding the
  remainder when the input integer is divided by 10. Add 48
  to this to get the ASCII code of the appropriate digit.
  Then use CHR to get the digit as a character. */
}

```

For future exercises, we will not give alternative implementations. In any question where you are asked for an implementation, there will be a number of possible correct answers.

Activity 2.3

An attempt to specify a function *DIGIT*, and code intended to implement it, are given below. The function *UNIT* as specified in Activity 2.2 is assumed to be available.

- Test the implementation with *number* equal to 21547 and with *n* equal to each of: (i) 1; (ii) 3; (iii) 6.
- Suggest how the specification might be modified to one that is correctly implemented by *impdigit*.

function *DIGIT*(*n in Int, number in Int*) **return in Char**

pre *n* > 0, *number* > 0.

post The returned value is the character giving the digit that appears in the *n*th place (counting from the right) in the expression of *number* as a decimal. For example, if *number* is 3472 and *n* is 3 then the value returned is '4'.


```

function DIGIT(n, number)
/* impdigit */
{
  var anInt in Int
  var aChar in Char
  var index in Int
  anInt <-- number
  for (index <-- 1 to n-1)
  {
    anInt <-- DIV(anInt,10)
  }
  aChar <-- UNIT(anInt)
  return aChar
}

```

Discussion

(a) (i) $n = 1$. In this case, the **for** statement becomes

for ($index \leftarrow 1$ **to** 0),

so the body of the **for** loop is never executed.

The assignment $anInt \leftarrow number$ sets $anInt$ to 21547. Then the assignment $aChar \leftarrow UNIT(anInt)$ sets $aChar$ to $UNIT(21547)$, which is the character '7'. This is the value returned by *impdigit* for these inputs.

Now the postcondition of *DIGIT* states that $DIGIT(1, 21547)$ is the character giving the digit appearing in the first place (from the right) in 21547. This is '7', so *impdigit* returns the correct value for these inputs.

(ii) $n = 3$. In this case, the **for** loop is executed with $index$ equal to 1 and 2.

Trace with inputs $n = 3$, $number = 21547$.

Comment	$index$	$anInt$	$aChar$
	-	21547	-
loop	1	2154	-
loop end	2	215	-
$aChar \leftarrow UNIT(anInt)$	-	215	'5'

The value returned is '5'. The postcondition of *DIGIT* states that the returned value should be the digit in the position 3 from the right in $number = 21547$. The third digit from the right is '5', so the correct value is returned in this case.

(iii) $n = 6$. In this case, the **for** loop is executed five times (with $index$ from 1 to 5).

Trace with inputs $n = 6$, $number = 21547$.

Comment	$index$	$anInt$	$aChar$
	-	21547	-
loop	1	2154	-
	2	215	-
	3	21	-
	4	2	-
loop end	5	0	-
$aChar \leftarrow UNIT(anInt)$	-	0	'0'

The value returned is '0'. The postcondition of *DIGIT* states that the returned value should be the digit in the position six from the right

in $number = 21547$. But there is no digit in the position six from the right! A returned value of '0' may seem reasonable in this case, but the postcondition should explain clearly what happens. So, to be strict, we would regard the specification as incomplete.

- (b) The slight mismatch between *impdigit* and the specification of *DIGIT* can be corrected in two ways. One is to modify the precondition, as below. This change makes sure that there are at least n digits in $number$ and so will remove the problem with the semantics.

pre $n > 0, number > 0, number \geq 10^{n-1}$.

Alternatively, the postcondition can be changed to spell out what happens in a case such as that in (a) (iii).

post The returned value is the character giving the digit that appears in the n th place (counting from the right) in the expression of $number$ as a decimal. For example, if $number$ is 3472 and n is 3 then the value returned is '4'. If there is no digit n places from the right (as will happen if $number < 10^{n-1}$), then the returned value is '0'.

You may wonder why the function *DIGIT* specified in Activity 2.3 might be of interest. Although we will not discuss how integers are represented within a computer, there is no reason to expect that this internal representation is based on the way we write them. The notation that we usually use, such as 3472, uses 10 as a basis for the way the number is written. The notation 3472 means that this number is made up by adding: 2; seven 10s; four 100s; and three 1000s; that is:

$$\begin{aligned} 3472 &= 3 \times 1000 + 4 \times 100 + 7 \times 10 + 2 \\ &= 3 \times 10^3 + 4 \times 10^2 + 7 \times 10^1 + 2 \times 10^0. \end{aligned}$$

$a^1 = a$ and $a^0 = 1$ for any number a .

Because of the way it is based on the number 10, this usual notation for writing numbers is referred to as **decimal** notation. (The notation for writing real numbers, using digits after a decimal point, is a natural extension of this.)

The representation of numbers within the computer is likely to be based on a number other than 10. For example, **binary** notation is based on the number 2. The number 13, for example, is

$$8 + 4 + 1 = \underline{1} \times 2^3 + \underline{1} \times 2^2 + \underline{0} \times 2^1 + \underline{1} \times 2^0.$$

In binary notation, 13 is expressed as 1101. (We take the multipliers of the powers of 2: these are underlined in the expression above.) Binary notation can express any number using just the two digits 0 and 1. These two values can be represented by a switch in an electrical circuit being open or shut. Thus this is the traditional basis for representing integers within computers.

If the representation of integers within the computer is different from the decimal notation that we normally use, then routines will be needed to translate between the internal representation of integers and decimal notation. This would be required before displaying numbers on a screen, or before printing them. Translation of the integer as represented within the machine into a sequence of digits from the set of characters {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'} might draw on a function such as *DIGIT*. Interpretation of an integer literal, presented as a string of digit characters, would require a similar process, in reverse.

For example:

Decimal	Binary
$0 = 0 \times 2^0$	0
$1 = 1 \times 2^0$	1
$2 = 1 \times 2^1 + 0 \times 2^0$	10
$3 = 1 \times 2^1 + 1 \times 2^0$	11

We refer to expressions that are not consistent with the rules of the course code as invalid. For example, the expression $CHR(1000)$ is not valid, because 1000 fails the precondition of CHR , while $7 \leq '8'$ is not valid, because there is no type function $\leq$ that compares an integer and a character. The simple interpretation of any piece of course code that is invalid is that the code does not work. However, in practice, a more precise interpretation of invalid code is that you cannot rely on what may happen. In particular, the WorkPad is an implementation of the course code — it is not identical to it. When an invalid expression is entered into the WorkPad, a variety of things may happen:

- the code may fail completely;
- the code may fail, and some error message be returned (which may — or may not — be informative);
- the expression may return some value.

If the last happens, the value returned will be unpredictable. For invalid code, there is no ‘correct’ value that can be returned, and so you have no way of telling what value might be returned by invalid code. For example, the expression $7 \leq '8'$ is invalid. (For the purposes of the course code, that is all we need to know. Such an expression should not be used in code.) In the WorkPad, this expression may fail, or it might return **false** — or it might return **true** (or 7 or ‘8’ — anything is possible).

Note that checking that code runs in the WorkPad is not a sufficient check of its validity.

Activity 2.4

What value should be returned by an implementation of the function *MAKEUPPER* when each of: (a) ‘e’; (b) ‘E’; (c) ‘a’; (d) ‘2’ is input?

function *MAKEUPPER*(*c* in Char) **return** in Char

pre The character *c* is a lower-case letter.

post The returned value is the letter *c* in upper case. So, for example, if *c* = ‘q’ then the returned value is ‘Q’.

Discussion Note that *MAKEUPPER* has the precondition that the input character *c* is a lower-case letter. Neither the input ‘E’ nor the input ‘2’ satisfies the precondition of *MAKEUPPER*.

- The value ‘E’ should be returned.
- The precondition of *MAKEUPPER* is not satisfied, so any value (or none) may be returned.
- The value ‘A’ should be returned.
- As in (b) any value (or none) may be returned.

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Express implementations in a form suitable for keying in to the WorkPad.
- ☐ Give implementations using the data types **Int**, **Bool** and **Char**.

Exercises on Section 2

Exercise 2.1

- (a) A code fragment is given below.

```
var aChar in Char
var num in Int
aChar <-- 'a'
num <-- ASC(aChar)
num <-- num + 1
aChar <-- CHR(num)
```

- (i) What is the state of the variable *aChar* after the code above has been executed?
- (ii) Suppose that in the first assignment, 'a' is changed to 'P'. What would be the final state of the variable *aChar* in that case?
- (b) Give a specification and implementation of a function *NEXTCH*, which inputs a character and returns the character whose ASCII code is 1 larger than that of the input character.
- (c) Give an implementation of the function *NEXT*, specified below. You may assume that the function *NEXTCH* is available, as well as the primitive data types.

function *NEXT*(*lower* in Char) **return** in Char

pre *lower* is a lower-case letter.

post If *lower* lies between 'a' and 'y', then the returned value is the letter following *lower* in alphabetical order. If *lower* = 'z', then the returned value is 'a'.

Exercise 2.2

The integer *n* is a **factor** of the integer *m* if it divides it exactly, so that the remainder when *m* is divided by *n* is 0. (For example, 5 is a factor of 15, but 5 is not a factor of 16.) An integer *p*, with *p* > 1, is **prime** if it has no factors (apart from 1 and *p* itself).

The first five prime numbers are: 2, 3, 5, 7 and 11.

- (a) The function *HASAFACT* inputs two integers, both greater than 1, and outputs **true** if the second integer is a factor of the first, and **false** otherwise. Give a specification and implementation of *HASAFACT*.
- (b) The function *ISPRIME* inputs an integer greater than 1, and returns **true** if it is prime, and **false** otherwise. Give a specification and implementation of *ISPRIME*. You may assume that *HASAFACT* is available. (Base your implementation on checking each number between 2 and *p* − 1 to see whether it is a factor of *p*.)

3 Arrays

In this section, we shall describe a data type called **Array of X**, whose form of data is a non-empty sequence of items. We refer to a variable of

type **Array of X** as an **array**. The items in an array will all come from the same data type. An array might be a sequence of integers, from *SeqOfInt*, or a sequence of characters, from *SeqOfChar*. We can describe functions that apply to sequences irrespective of the type of items that they contain. In the specifications of the type functions, we will use sequences whose items are of a general, unspecified, type, represented by **X**.

The data set of **Array of X** does not include a sequence of length 0. Arrays are used for storing data, and an array with no spaces in it is not likely to be useful. In **Array of X**, **X** gives the type of items in the sequence, and so it could be **Char** or **Int**, for example.

Array of X has three type functions: *SIZE*, *PUT* and *AT*. The first of these, *SIZE*, gives the number of items in the sequence (that is, the length of the sequence). The second type function, *PUT*, allows you to place an item in the array at a specified position. So *PUT*(*i*, *s*, *x*) is the array formed from the array *s* by replacing the item at position *i* with an input item *x*.

The items in an array are indexed starting with index 1, and counting from the left. So, for example, the second item in the array [6, 11, 4, 5, -7] is the integer 11. The value *PUT*(2, [6, 11, 4, 5, -7], 100) is obtained by changing the value at the position with index 2 to the value input as *x*, which here is 100. We have, for example:

$$\begin{aligned} PUT(2, [6, 11, 4, 5, -7], 100) &= [6, 100, 4, 5, -7]; \\ PUT(4, ['a', 'b', 'c', 'd', 'e'], 'Q') &= ['a', 'b', 'c', 'Q', 'e']. \end{aligned}$$

Once an array variable has been declared in code, the number of items that it contains cannot be changed. To allocate storage for an array variable, a machine needs to know two things. These are the type of item to be stored in the array, and the size of the array. The course code deals with these two features together. For example, to create an array variable *myArr* to store five characters, we would declare it like this:

```
var myArr in Array (of size 5) of Char
```

When an array has just been declared, it has nothing stored in it. It can be filled in two ways. The first is to assign an array literal of the correct size, as in, for example:

```
var myArr in Array (of size 5) of Char
myArr <-- ['a', 'b', 'c', 'd', 'e']
```

In the course code, we write an array literal, as in [2, 3, 5, 7, 11], for example, using the standard notation for a sequence (square brackets [and]). Such literals can be used in an assignment statement, as was done above. Each primitive type that you have met has its own distinct notation for writing literals. The notation used in code enables the type of any literal to be immediately apparent. The computer needs to recognise the data type of any literal, and to have a mechanism for translating the notation used in code into whatever electronic representation is used for variables of that type. We shall not discuss such mechanisms.

In Unit 1, we used a notation involving double inverted commas as a shorthand for sequences of characters. For example, we used “Word” to mean ['W', 'o', 'r', 'd']. However, to show a literal value from **Array of Char** we

We will give specifications of these functions shortly. You met them in Unit 1.

In many programming languages, the index starts at 0 rather than 1.

must use the full sequence notation. The double inverted comma notation cannot be used for a literal from **Array of Char**.

Sometimes, one wants to fill up an array by repeated use of *PUT*. When an array is filled in this way, there will be a number of lines of code before the array is filled. During this process, the array will contain some unfilled spaces, which we will refer to as **blanks**. To write traces of code involving array variables, we will need a notation to show blanks in arrays. An example of how this will be done is given below. You will see that $\perp$ is used to show blanks in the array *arr*.

Read $\perp$ as “blank” in this context.

Code	<i>anArr</i>
var <i>anArr</i> in Array (of size 3) of Char	$[\perp, \perp, \perp]$
<i>anArr</i> $\leftarrow$ <i>PUT</i> (1, <i>anArr</i> ,‘a’)	['a', $\perp$, $\perp$]
<i>anArr</i> $\leftarrow$ <i>PUT</i> (2, <i>anArr</i> ,‘b’)	['a', ‘b’, $\perp$]
<i>anArr</i> $\leftarrow$ <i>PUT</i> (3, <i>anArr</i> ,‘c’)	['a', ‘b’, ‘c’]

The notation $\perp$ for a blank is for use in writing traces; it is not something that the WorkPad is able to interpret.

The final type function of **Array of X** is *AT*. The function *AT* retrieves an item from an array at a specified location. We need to be careful in specifying *AT*, since it is not apparent what should be returned if the array contains a blank at the specified location! For convenience, we shall set a precondition on *AT* requiring that there are no blanks in the input array before *AT* can be called. This avoids the possibility of *AT* looking in a location that is blank for a value to return. We will refer to an array containing no blanks as **full**.

Consider Fragment 1, below.

```
/* Fragment 1 */
var myArr in Array (of size 5) of Char
myArr <-- ['a','b','c','d','e']
```

After Fragment 1 is executed, the state of the variable *myArr* is ['a','b','c','d','e']. Subsequent code can change the state of *myArr*, but it cannot change either the type of item stored in the array, or the size of the array. Once an array variable has been created, you can think of it as a fixed number of pigeonholes, each with an associated index, and each of which can store an item of the specified type. (See Figure 3.1.)

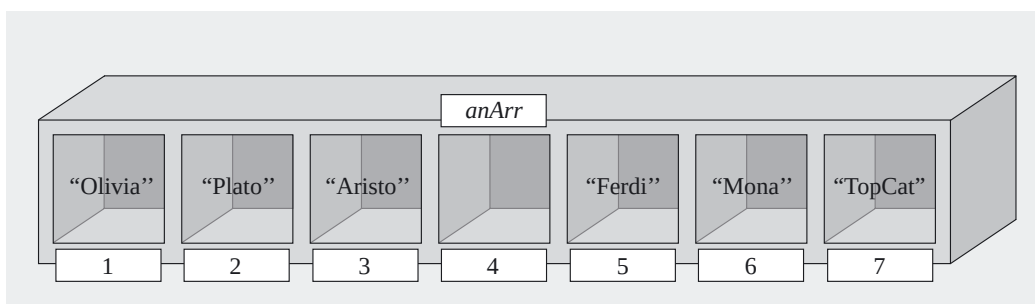


Figure 3.1 A variable *anArr* storing an array giving the names of the cats currently resident in an Oxford cattery. Each guest’s name is stored as a string. There is accommodation for 7 cats, but residence 4 is currently unoccupied. (The variable *anArr* would be an array of strings. We will describe a type whose form of data is strings in a later unit.)

After the code in Fragment 1 above has been executed, $SIZE(myArr)$ will return the value 5. We could call $AT(anInt, myArr)$ with $anInt$ any of 1, 2, 3, 4 or 5 (but not any other value), and we could call $PUT(anInt, myArr, item)$ for the same values of $anInt$, and with $item$ any character.

We can now give a full description of the data type **Array of X**. The specifications of the type functions take account of the possibility that arrays may contain blanks.

Array of X	Data type
<i>Form of data</i> An array is a non-empty sequence of elements whose length is fixed. Some places in the array may be unfilled (blank). <i>Type functions</i> function $SIZE(arr \text{ in Array of X})$ return in Int function $AT(i \text{ in Int}, arr \text{ in Array of X})$ return in X function $PUT(i \text{ in Int}, arr \text{ in Array of X}, x \text{ in X})$ return in Array of X	

In traces, an array is written as an array literal, and with blanks shown as $\perp$. We will say that an array is full if it contains no blanks.

function $SIZE(arr \text{ in Array of X})$ **return in** Int
pre true.
post The returned value is the number of items in the sequence arr . Blanks are counted.

function $AT(i \text{ in Int}, arr \text{ in Array of X})$ **return in** X
pre The array arr is full and $1 \leq i$ and $i \leq SIZE(arr)$.
post The returned value is the i th item in the sequence arr .

function $PUT(i \text{ in Int}, arr \text{ in Array of X}, x \text{ in X})$ **return in** Array of X
pre $1 \leq i$ and $i \leq SIZE(arr)$.
post The returned value is an array with x as its i th item, and otherwise identical to arr .

If you have met arrays in some particular programming language, then you may be surprised by some aspects of the way in which we treat arrays here. Individual programming languages handle arrays in different ways. A common feature is that an array variable is a sequence of fixed size — code cannot change the number of items stored in an array.

Activity 3.1

- What is $AT(3, [7, 11, 32, 9])$?
- Let $s = ['a', 'b', 'c', 'd']$. What are:
 - $SIZE(s)$; (ii) $AT(2, s)$; (iii) $PUT(2, s, 'c')$?
- Write down an expression that gives the first (left-most) item of an array arr .

Discussion

- (a) This is 32 — the item in the third position in the array [7, 11, 32, 9].
- (b) (i) There are four items in ['a','b','c','d'], and so $SIZE(s) = 4$.
 (ii) The second item in ['a','b','c','d'] is the character 'b', so $AT(2, s) = \text{'b'}$.
 (iii) Replacing the second item in ['a','b','c','d'] with 'c', gives us $PUT(2, s, \text{'c'}) = [\text{'a'}, \text{'c'}, \text{'c'}, \text{'d'}]$.
- (c) The first item of an array has index 1. So the required item is given by $AT(1, arr)$.

Activity 3.2

A variable *aVar* is declared and initialised as below.

```
var aVar in Array (of size 6) of Char
aVar <-- ['a','b','c','d','e','f']
```

- (a) After this code is executed, what value will be returned by each of:
 (i) $SIZE(aVar)$; (ii) $AT(3, aVar)$; (iii) $PUT(2, aVar, \text{'z'})$?
- (b) Explain why each of the following expressions is not valid:
 (i) $AT(8, aVar)$; (ii) $PUT(2, aVar, 7)$.

Discussion

- (a) (i) 6. (ii) 'c'. (iii) ['a','z','c','d','e','f'].
- (b) (i) $AT(8, aVar)$ is invalid because *aVar* only contains six items, and so there is no item in position 8 in *aVar*. (This is reflected in the precondition for *AT*, which includes the condition $i \leq SIZE(s)$. Here $SIZE(s) = 6$. Then with $i = 8$, this condition becomes $8 \leq 6$, which is **false**.) (ii) $PUT(2, aVar, 7)$ is invalid because *aVar* is an array containing characters, and 7 is an integer, not a character.

The data type **Array of X** is another primitive type in the course code. As with the types in Section 1, we will not discuss how **Array of X** might itself be implemented on a computer.

Within an array, all the stored items are of the same type. The **X** in **Array of X** is a single data type such as **Int**.

A data type such as **Array of X** is known as a **parameterised** type. This is because of the parameter **X**, which may itself be any type. For example, **X** might be any of the types you met in Section 1. Thus we can declare a variable to be of type **Array of Int**, or of type **Array of Char**, or of type **Array of Bool**. In the examples you will meet in this unit, **X** will either be one of the types **Int**, **Bool** or **Char**, or will be an unspecified type **X**.

In principle, **X** may be any data type. For example, **X** might be **Array of Char**. So we can have a variable of type **Array of Array of Char**.

In practice, it would not be possible for a computer to store arrays of an arbitrary size. There would need to be a largest acceptable size of array (and this would depend on the form of data being stored in the array). Even if we were to incorporate this constraint into our code, it would not affect the preconditions required for the type functions, since these do not change the size of the array. For simplicity, we shall not give a bound on array size.

Not every programming language behaves like this. This is another feature of strongly typed programming languages.

Some authors call a type such as **Array of X** a **generic type**, and refer to the unspecified type **X** here as a **formal generic parameter**.

Activity 3.3

Trace the values taken by the variables in each of the code fragments below.

```
/* FragA */
var anArr in Array (of size 5) of Int
var bArr in Array (of size 5) of Int
var num in Int
anArr <-- [5,4,3,2,1]
num <-- AT(2,anArr)
num <-- 2 * num
bArr <-- PUT(2,anArr,num)
```

```
/* FragB */
var anArr in Array (of size 3) of Char
anArr <-- PUT(2,anArr,'a')
anArr <-- PUT(3,anArr,'b')
anArr <-- PUT(1,anArr,'c')
```

Discussion Traces are given below.

Code for FragA	<i>anArr</i>	<i>num</i>	<i>bArr</i>
<i>anArr</i> ← [5, 4, 3, 2, 1]	[5,4,3,2,1]	-	[⊥, ⊥, ⊥, ⊥, ⊥]
<i>num</i> ← AT(2, <i>anArr</i>)	[5,4,3,2,1]	4	[⊥, ⊥, ⊥, ⊥, ⊥]
<i>num</i> ← 2 * <i>num</i>	[5,4,3,2,1]	8	[⊥, ⊥, ⊥, ⊥, ⊥]
<i>bArr</i> ← PUT(2, <i>anArr</i> , <i>num</i>)	[5,4,3,2,1]	8	[5,8,3,2,1]

Code for FragB	<i>anArr</i>
var <i>anArr</i> in Array (of size 3) of Char	[⊥, ⊥, ⊥]
<i>anArr</i> ← PUT(2, <i>anArr</i> , 'a')	[⊥, 'a', ⊥]
<i>anArr</i> ← PUT(3, <i>anArr</i> , 'b')	[⊥, 'a', 'b']
<i>anArr</i> ← PUT(1, <i>anArr</i> , 'c')	['c', 'a', 'b']

Note that in FragA *anArr* does not change at all. To change *anArr*, the last line of code should be
anArr ← PUT(2, *anArr*, *num*)

In FragB, however, we have used such assignment statements, for example
anArr ← PUT(2,*anArr*, 'a')
and so *anArr* does change.

Any array variable is created with two features. The array contains items of a particular type. Also, it has a fixed size. These two features of an array variable are set by the **var** statement, and cannot be changed. Consequently, one needs to be careful when making an assignment to an array variable. In the course code, assignment of arrays must be consistent with both these features of the array variable. So it must satisfy the following rule:

An assignment of arrays $s \leftarrow t$ is only valid if both the conditions below hold.

1. The arrays s and t contain items of the same type.
2. The arrays s and t have the same size.

The features of the array variable s will be apparent from its **var** statement. If t is another array variable, both the features in its **var** statement (size and type of entry) must match those for s , before an assignment $s \leftarrow t$ is made. If t is an array literal, the notation used will make the type of entry apparent, and again both item type and size must match these features of s .

This rule about assignment may look as if it will be difficult to work with. However, a value of the form $PUT(place, anArr, item)$ is bound to be an array of the same size as $anArr$. Once an array variable has been created, our code will typically use assignments of the form:

$$anArr \leftarrow PUT(place, anArr, item).$$

With this assignment, all that you need to check is that you are putting an $item$ of the correct type into the array, and that $place$ is consistent with the size of $anArr$.

Activity 3.4

Suppose that the variable $aVar$ has been declared in the statement:

var $aVar$ in Array (of size 3) of Int

Which of the assignments in (a)–(c) is valid?

- (a) $aVar \leftarrow PUT(1, aVar, 8);$
- (b) $aVar \leftarrow PUT(8, aVar, 1);$
- (c) $aVar \leftarrow PUT(3, aVar, 'a').$

Discussion

- (a) This assignment is valid. $aVar$ is an array containing integers, and so 8 is of the correct type to PUT into $aVar$. Also, $aVar$ has size 3, and so 1 satisfies the precondition of PUT , which here is $(1 \leq 1 \text{ and } 1 \leq 3)$.
- (b) This assignment is not valid. With these inputs the precondition of PUT is **false**. (It is $(1 \leq 8 \text{ and } 8 \leq 3)$, and $8 \leq 3$ is **false**.)
- (c) This assignment is not valid. In this case, $aVar$ contains integers, so 'a' (which is a character) is of the wrong type to PUT into $aVar$.

Activity 3.5

- (a) Write down an expression that gives the last (right-most) item of an array arr .
- (b) Give a specification and implementation of a function $LAST$ that returns the last (right-most) item in an array.
- (c) What values should your implementation return when the input is:
 - (i) $[11, 22, 33, 44]$; (ii) $['a', 'b']$?

Discussion

- (a) The last item in the array arr will have index $SIZE(arr)$. So the last item is given by $AT(SIZE(arr), arr)$.
- (b) A specification and implementation are given below. Notice that you need to include " arr is full" in the precondition so that the function AT can be called. This is an example where it is convenient to return an expression. We use the expression suggested by the answer to part (a).

function $LAST(arr \text{ in Array of } X)$ return in X

pre arr is full.

post The returned value is the last item in the array arr .

```
function LAST(arr)
{
  return AT(SIZE(arr),arr)
}
```

- (c) The returned values should be: (i) 44; (ii) 'b'.

Code to process all the elements in an array will typically use a **for** loop, with a loop variable corresponding to the position of each item within the array. For convenience, we will normally only consider functions that input full arrays. (So the specifications will include a precondition that the input array is full.)

Activity 3.6

A specification and implementation of a function *SUM* are given below.

- (a) What value should be returned by the implementation for each of the following inputs: (i) [11, 22, 33, 44] (ii) [5, 2, 7, 11]; (iii) [4, -7, -11, 3, 1]; (iv) [0, 0, 0]?
- (b) Trace the implementation when the array [11, 22, 33, 44] is input. Is the correct value returned for this input?

function *SUM*(*arr* in Array of Int) **return** in Int

pre *arr* is full.

post The returned value is the sum of the numbers in the sequence *arr*.

```
function SUM(arr)
{
  var sum in Int
  var index in Int
  sum <-- 0
  for (index <-- 1 to SIZE(arr))
  {
    sum <-- sum + AT(index,arr)
  }
  return sum
}
```

Discussion

- (a) The values that should be returned are determined by looking at the postcondition of *SUM*. They are:
- (i) $11 + 22 + 33 + 44 = 110$;
 - (ii) $5 + 2 + 11 + 7 = 25$;
 - (iii) $4 - 7 - 11 + 3 + 1 = -10$;
 - (iv) $0 + 0 + 0 = 0$.
- (b) The array [11, 22, 33, 44] has size 4, and so in the **for** loop, *index* takes the values 1, 2, 3 and 4. A trace is given below.

Trace with input [11, 22, 33, 44]

Comment	<i>index</i>	$AT(index, arr)$	<i>sum</i>
$sum \leftarrow 0$	-	-	0
loop	1	11	11
	2	22	33
	3	33	66
loop end	4	44	110

The value returned is 110. This is correct (see (a) (i)).

EXAMPLE 3.1

The function *DOUBLE* inputs an array of integers, and outputs the array formed by doubling each of the entries. So, for example,

$$DOUBLE([2, 1, -4, 7]) = [4, 2, -8, 14].$$

Suppose that we want to implement *DOUBLE*. To do this, we can read a number from the input array, say *arr*, double it, then return this doubled value to the same position in an array holding the output values. To create an array to hold the output values, we can set up a blank array of the appropriate size. This will need to be of the same size as *arr*, that is, $SIZE(arr)$.

We give below a specification and implementation for *DOUBLE*.

function *DOUBLE*(*arr* in Array of Int) return in Array of Int

pre *arr* is full.

post The returned value is an array whose members are the numbers in *arr*, with each multiplied by 2. So, for example, $DOUBLE([2, 1, -4, 7]) = [4, 2, -8, 14]$.

```
function DOUBLE(arr)
{
  var newArr in Array (of size SIZE(arr)) of Int
  var num in Int
  var index in Int
  for (index <-- 1 to SIZE(arr))
  {
    num <-- AT(index, arr)
    /* Extracts as num the number with position index
    in the given array. */
    num <-- 2 * num
    /* Doubles num. */
    newArr <-- PUT(index, newArr, num)
    /* Puts num (now double the original number) into
    the new array, newArr, at the same position (index) that
    it had in arr. */
  }
  return newArr
}
```

Consider the function *ADDLAST*, whose specification is given below.

function *ADDLAST*(*x* in **X**, *arr* in **Array of X**) **return** in **Array of X**

pre *arr* is full.

post The returned array has size one greater than *arr*. It is formed by adding *x* to the array *arr*, so that *x* is the last item in the returned array. For example, *ADDLAST*(1, [2, 3, 4, 5, 6]) = [2, 3, 4, 5, 6, 1].

ADDLAST returns an array whose size is different to that of the input array. To give an implementation of *ADDLAST*, we need first to determine the size of the array that will be returned by *ADDLAST*, and then to declare an array variable of that size. The size of the returned array will depend on the size of the input array. We can see that the returned array will contain one more item than the input array. So the array *ADDLAST*(*x*, *arr*) has size *SIZE*(*arr*) + 1.

We can then implement *ADDLAST* as follows.

1. Declare an array variable of size *SIZE*(*arr*) + 1 (to hold the array to be returned).
2. Copy each item from the input array *arr* to this array variable (without changing the position of the item in the array).
3. Then add *x* at the end of the array variable.

Activity 3.7

Give an implementation of the function *ADDLAST*, specified above.

Discussion

```
function ADDLAST(x,arr)
{
  var newArr in Array (of size SIZE(arr) + 1) of X
  var item in X
  var index in Int
  for (index <-- 1 to SIZE(arr))
  {
    item <-- AT(index,arr)
    newArr <-- PUT(index,newArr,item)
  }
  /* Copies each item from arr to the same position
  in newArr. */
  newArr <-- PUT(SIZE(arr) + 1,newArr,x)
  /* Puts x at the end of newArr. */
  return newArr
}
```

Suppose that *tillSeq* is a variable of type **Array of Int**, and *nextItem* is a variable of type **Int**. We could not then use the statement

```
tillSeq <-- ADDLAST(nextItem,tillSeq)
```

in code. That is because *ADDLAST*(*nextItem*, *tillSeq*) has size 1 greater than the previous value in *tillSeq*, and so this assignment is not valid. It attempts to assign an array of the wrong size to *tillSeq*.

If we wish to handle data whose structure is a sequence whose size may vary, we need a new data type. This needs to be based on a sequence whose size is not fixed. Such a data type is described in the next section.

Aside The data set of arrays

It is a little awkward to make precise the data set underlying the data type **Array of X**, since we need to take account of the possibility that the array may contain blanks. An array is a sequence of items that ‘usually’ come from the set X (the data set underlying the type **X**), but may be blank. So an item in the array comes from the set $X \amalg \{\perp\}$. Hence the set of possible states of a variable of type **Array of X** is $\text{SeqOf}(X \amalg \{\perp\})$.

Read $\amalg$ as “or” — see Unit 1.

The mathematical functions corresponding to the type functions have the following signatures:

$$\begin{aligned} \text{SIZE} &: \text{SeqOf}(X \amalg \{\perp\}) \rightarrow \text{Int}; \\ \text{PUT} &: \text{Int} \times \text{SeqOf}(X \amalg \{\perp\}) \times X \rightarrow \text{SeqOf}(X \amalg \{\perp\}); \\ \text{AT} &: \text{Int} \times \text{SeqOf}(X \amalg \{\perp\}) \rightarrow X. \end{aligned}$$

It would be inconvenient for the output set of AT to be $(X \amalg \{\perp\})$, because then any process handling the output would need to deal with the possibility of a blank value. That problem was avoided by placing a precondition on AT .

End of Aside

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Recognise and use the terminology: array; parameterised (data type); blank (in an array).
- ☐ Interpret and use the notation associated with the data type **Array of X**, including:
 - the type functions SIZE , AT , and PUT ;
 - use of sequence notation for array literals;
 - $\perp$ to represent blanks in arrays when tracing array variables in code;
 - the notation for declaring an array variable.
- ☐ Recognise when an assignment to an array variable is valid, taking account of the need for the arrays to be of the same size, and to contain members of the same type.
- ☐ Interpret a specification involving the type **Array of X**.
- ☐ Trace code involving array variables, and test the returned value against a given specification.
- ☐ Give implementations using **Array of X** (and the data types **Int**, **Bool** and **Char**).

Exercises on Section 3

Exercise 3.1

If $\text{arr} = [21, 22, 23, 24, 25, 26]$, what is each of:

- (a) $\text{AT}(4, \text{arr})$;
- (b) $\text{PUT}(5, \text{arr}, 1)$;
- (c) $\text{PUT}(1, \text{arr}, 5)$?

Exercise 3.2

- (a) Write an expression that gives the second element in an array *arr*.
- (b) Give the value of your expression in (a) when *arr* is:
 - (i) [5, 7, 22, -13, 0, 2]; (ii) ['a', 'r', 'r', 'a', 'y'].
- (c) Give a specification and implementation of a function *ARRSEC* that returns the second element in an array.

Exercise 3.3

- (a) Give an implementation of the function *COUNTOF* specified below. You can assume that the data type **X** has a test of equality `==`.

function *COUNTOF*(*item* in **X**, *arr* in Array of **X**) **return** in **Int**
pre *arr* is full.
post The returned value gives the number of times that *item* appears in *arr*. If the item does not appear at all in *arr*, then the returned value is 0.

- (b) Test your implementation when the inputs are 'a' and ['a', 'a', 'r', 'd', 'v', 'a'].

4 Stacks

An array variable has a fixed size. This restriction means that there are circumstances in which it is inconvenient to store data in an array. For example, one cannot know in advance how many items there will be in a sequence of items entered at a supermarket check out. The number of items in a sequence variable that stores the items entered so far will increase by one as each new item is entered.

In this section we describe a new data type, called **Stack of X**, whose form of data is a sequence whose size is not fixed. Variables of this type are called **stacks**. In a trace we will write a stack of items with the usual notation for a sequence, such as [5, 1, 9]. For a stack, it is helpful to picture the items in the sequence as forming a pile, with the first item in the sequence at the bottom of the pile. The item written at the right of the sequence is at the top of the pile. (See Figure 4.1.) The type functions on stacks only allow you to work at the top of this pile. You can examine the top item (using the function *PEEK*), remove the top item (using *POP*), or add an item at the top of the pile (using *PUSH*).

As with arrays, the items in a stack are of an unspecified type **X**, and we do not mix types within a stack. So **Stack of X** is another parameterised data type, and **X** might be any type, such as **Char**, **Int** or **Bool**. Unlike arrays, the empty sequence is permitted as a possible state of a stack variable.

The data type **Stack of X** introduces a new feature. It has a **default** value (which is the empty sequence). When a stack variable is declared, it is created with this default value. The types introduced earlier in this unit do not have default values, but data types that you will meet later will usually have defaults. In traces, we will write the state of a stack variable using sequence notation. (When giving examples of the effect of a function

The functions *PUSH* and *POP* on stacks have the same effect as the functions *ADDLAST* and *REMOVELAST* that you used in Unit 2.

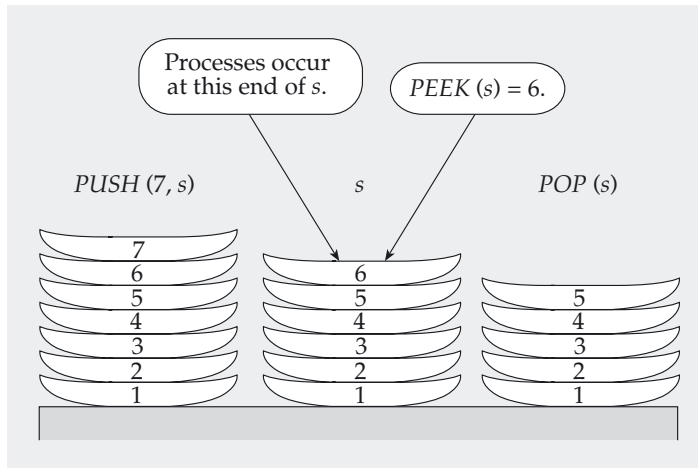


Figure 4.1 A stack pictured as a pile of items. You can add an item only to the top of pile, and you can examine or discard only the item at the top of the pile. With $s = [1, 2, 3, 4, 5, 6]$, we have: $PEEK(s) = 6$; $POP(s) = [1, 2, 3, 4, 5]$; $PUSH(7, s) = [1, 2, 3, 4, 5, 6, 7]$.

specification, we will also write the state of a variable of stack type using sequence notation.) In traces of variables with type **Stack of Char**, we may use the shorthand notation “abc”.

We now give the specification of **Stack of X**.

We can use shorthand notation like “abc” for stacks since there are no blanks in a stack. We do not use this notation when tracing an array of characters, because it might have blanks.

Stack of X

Data type

Form of data

A sequence of elements, whose size may vary. This can be written in the usual sequence notation, for example as $[1, 4, 22, -3]$.

Default The sequence is empty.

Type functions

function $SIZE(s \text{ in Stack of X})$ **return in** Int

function $PUSH(x \text{ in X}, s \text{ in Stack of X})$ **return in** Stack of X

function $POP(s \text{ in Stack of X})$ **return in** Stack of X

function $PEEK(s \text{ in Stack of X})$ **return in** X

function $SIZE(s \text{ in Stack of X})$ **return in** Int

pre true.

post The returned value is the number of items in the sequence s .

function $PUSH(x \text{ in X}, s \text{ in Stack of X})$ **return in** Stack of X

pre true.

post The returned value is the stack formed by adding x to the top of s . So, for example, $PUSH(5, [1, 4, -2]) = [1, 4, -2, 5]$.

function $POP(s \text{ in Stack of X})$ **return in** Stack of X

pre s contains at least one item.

post The returned value is the stack formed by deleting the item from the top of s . So, for example, $POP([1, 4, -2]) = [1, 4]$.

function $PEEK(s \text{ in Stack of X})$ **return in** X

pre s contains at least one item.

post The returned value is the item at the top of s . So, for example, $PEEK([1, 4, -2]) = -2$.

Note that the top item in the stack is the last item in the sequence giving the stack items.

In practice, stacks will require a more flexible approach to allocation of storage than is needed for arrays. To make the type **Stack of X** available, we need a way to store a sequence whose size may vary. The approach to storage might be based on pairs (cell, link), where the link shows the storage area for the next pair, and the cell stores the sequence item. An extra pair would be added each time an item is added to the stack. Alternatively, storage might be based on arrays of a set size, where a new array and a link to it is added when each array is filled. However, details of how storage is allocated for a stack will not concern us in this course. We will assume that we have a way of storing stacks without giving details.

Storage limitations would mean that there would need to be a bound on the size of a stack that could be allowed in practice. So in a programming language, a stack data type would need to include an upper bound on the size of stack permitted. This would require a precondition on *PUSH*, to avoid forming a stack that was too large. As with similar issues for **Int**, we shall not give a specific bound on stack size, in order to avoid complications.

Activity 4.1

Suppose that *aStack* is a variable of type **Stack of Char**, whose state is ['a','b','c','d']. What are each of:

- (a) *PEEK*(*aStack*);
- (b) *POP*(*aStack*);
- (c) *SIZE*(*aStack*);
- (d) *PUSH*('Q',*aStack*)?

Discussion

- (a) The top item in ['a', 'b', 'c', 'd'] is at the right-hand end of the sequence. So *PEEK*(['a', 'b', 'c', 'd']) = 'd'.
- (b) *POP*(['a', 'b', 'c', 'd']) = ['a', 'b', 'c'].
- (c) *SIZE*(['a', 'b', 'c', 'd']) = 4.
- (d) *PUSH*('Q',['a', 'b', 'c', 'd']) = ['a', 'b', 'c', 'd', 'Q'].

Activity 4.2

Trace the values of the variables when the code fragment given below is executed.

```
var aStack in Stack of Int
var index in Int
for (index <-- 1 to 4)
{
    aStack <-- PUSH(index, aStack)
}
```

Discussion Remember that a stack variable is created with the default value given for the data type **Stack of X**. This is the empty sequence. A trace is given below.

Comment	<i>index</i>	<i>aStack</i>
var aStack in Stack of Int	-	[]
loop	1	[1]
	2	[1, 2]
	3	[1, 2, 3]
loop end	4	[1, 2, 3, 4]

Activity 4.3

- (a) Suppose that a variable *aStack* of type **Stack of Int** is in the state $[3, 11, -4, 0, 5, 6]$ when the code below is encountered. What is the state of *aStack* after the code is executed?

```
aStack <-- POP(aStack)
aStack <-- POP(aStack)
```

- (b) If *aStack* is a stack variable in the state $[3, 11, -4, 0, 5, 6]$, what is $PEEK(POP(aStack))$?

Discussion

- (a) The first statement deletes the top number from the stack, which is the last number in the sequence. This leaves *aStack* in the state $[3, 11, -4, 0, 5]$. The second statement deletes the top element of this, leaving *aStack* in the state $[3, 11, -4, 0]$.
- (b) $PEEK(POP([3, 11, -4, 0, 5, 6])) = PEEK([3, 11, -4, 0, 5]) = 5$.

Activity 4.4

Give a specification and implementation of a function *SECOND* that returns the item second from the top in a stack. (So for example, $SECOND([1, 2, 0, 7, 5]) = 7$.)

Discussion

We give a specification and implementation below.

function *SECOND*(*s* in **Stack of X**) **return** in **X**

pre *s* contains at least two items.

post The returned value is the item second from the top in *s*. For example, $SECOND([1, 2, 0, 7, 5]) = 7$.

```
function SECOND(s)
{
  var myStack in Stack of X
  myStack <-- s
  myStack <-- POP(myStack)
  return PEEK(myStack)
}
```

The code `return PEEK(POP(s))` would provide an alternative implementation of *SECOND*. You might be tempted to give `return AT(2, s)` as an implementation. However, this is not correct. The input *s* to *SECOND* is stated in the signature line to be of type **Stack of X**. So the function *AT* cannot be applied to *s*, since *AT* requires an array, not a stack, as input. (We write the states of variables of both stack and array type using the same sequence notation; so they look the same, but they are different to a machine, since they will be stored in different ways.) Although we use notation like `[1, 2, 3]` to show the state of a stack variable in traces, note that in code `[1, 2, 3]` is not a literal of type **Stack of Int**. We use notation such as `[1, 2, 3]` in code for a literal of type **Array of Int**. Consequently, this notation cannot also be used for a stack literal.

Activity 4.5

Suppose that you want a function *DELSEC* that deletes the item second from the top of a stack. (So, for example, *DELSEC*(`[5, 7, 11, 0, 2]`) = `[5, 7, 11, 2]`.) This can be implemented with the following sequence of actions.

1. Save the top item in the stack in a local variable (using *PEEK*).
2. Initialise a variable *newStack*, as the input stack.
3. Delete the top two items from *newStack* (using *POP*).
4. Replace the saved top item (using *PUSH*).

Give a specification and implementation of the function *DELSEC*.

Discussion A specification and implementation are given below.

function *DELSEC*(*s* in **Stack of X**) **return** in **Stack of X**

pre The stack *s* contains at least two items.

post The returned value is formed by deleting from *s* the item second from the top.

```
function DELSEC(s)
{
  var newStack in Stack of X
  var anX in X
  newStack <-- s
  anX <-- PEEK(s)
  /* The top item in s is stored in anX. */
  newStack <-- POP(newStack)
  /* newStack is the stack formed from s by deleting the
  top item. */
  newStack <-- POP(newStack)
  /* Deletes the item second from top of s from newStack. */
  newStack <-- PUSH(anX, newStack)
  /* Replaces the top item from s on top of newStack. */
  return newStack
}
```

Activity 4.6

Below is a specification of a function *REVERSE* and implementation *revstack*.

- (a) What value should be returned by *revstack* for each of the following inputs: (i) [5, 7, 2, 11]; (ii) ['a', 'b', 'c']; (iii) []?
- (b) Trace *revstack* in order to determine the value it returns when the stack ['a', 'b', 'c'] is input. Is the returned value correct in this case?

function *REVERSE*(*s* in Stack of **X**) **return** in Stack of **X**

pre true.

post The returned value is the stack formed by placing the items in *s* in reverse order. (If *s* is empty, the empty stack is returned.)

```
function REVERSE(s)
/* revstack */
{
  var old in Stack of X
  var newStack in Stack of X
  var item in X
  var index in Int
  old <-- s
  for (index <-- 1 to SIZE(s))
  {
    item <-- PEEK(old)
    old <-- POP(old)
    newStack <-- PUSH(item,newStack)
  }
  return newStack
}
```

Discussion

- (a) The values that should be returned are determined by looking at the postcondition of *REVERSE*. (i) [11, 2, 7, 5]; (ii) ['c', 'b', 'a'], (iii) [].
- (b) A trace is given below. With *s* = "abc", we have *SIZE*(*s*) = 3. So the **for** loop is repeated with *index* equal to 1, 2 and 3. Remember that a stack variable is initialised with the default value, which is an empty sequence. The value returned is "cba". This is the correct value for this input (see (a) (ii)).

Trace with input "abc"

	<i>index</i>	<i>item</i>	<i>old</i>	<i>newStack</i>
<i>old</i> ← <i>s</i>	-	-	"abc"	[]
loop	1	'c'	"ab"	"c"
	2	'b'	"a"	"cb"
loop end	3	'a'	[]	"cba"

Note that you may use the shorthand notation for sequences of characters in the trace.

Implementations based on the data type **Stack of X** have a different flavour from those using **Array of X**, because the type functions on stacks only let you work at the top of the stack. You can work through an array of data using an integer variable that gives the position of each item in turn. Implementations using stacks typically involve a variable stack, with each item dealt with when it is the top item, and then removed using *POP* after it has been dealt with.

It is sometimes useful to be able to merge two sequences to form a single sequence. This process is called **concatenation**. To concatenate the sequences s and t , first write the members of s , then the members of t . (The resulting single sequence is often written as $s \oplus t$.) We can specify this process, applied to stacks, as below.

function *CONCAT*(s in Stack of X , t in Stack of X) **return** in Stack of X
pre true.

post The returned value is the sequence formed by writing first the members of s , then the members of t . So for example,
CONCAT([1, 2], [6, 7, 8]) = [1, 2, 6, 7, 8] and *CONCAT*("he", "at") = "heat".

Now suppose that we want to implement the function *CONCAT*. To do this, one needs to add the members of the sequence t to the sequence s . This will have to be done one item at a time, using *PUSH*. Items will be added at the top of the stack, which is the right-hand end of the sequence. In the first example above, we want to add the items from the stack [6, 7, 8] in the order: first 6, then 7, then 8. But we can only see one item in a stack (using *PEEK*). This is the top, or rightmost, item. So we need first to reverse the order of the items in t .

A strategy for an implementation could be as follows.

1. Declare two local variables of stack type — say *firstPart* and *toAdd*.
2. Initialise *firstPart* as s , and *toAdd* as t .
3. Reverse *toAdd*.
4. Repeat the actions (once for each item in t):
 add the top item in *toAdd* to the top of *firstPart*;
 delete the top item of *toAdd*.
5. Return *firstPart*.

For example, with $s = \text{"he"}$ and $t = \text{"at"}$, this strategy works as below.

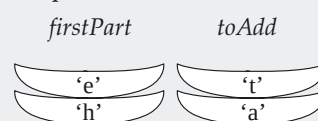
Activity 4.7

Give an implementation of *CONCAT*, specified above. As well as the type functions, you can assume that the function *REVERSE* (see Activity 4.6) is available.

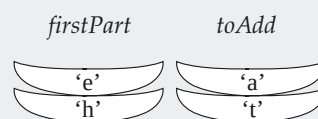
Discussion We will use the strategy outlined above.

```
function CONCAT(s,t)
{
  var firstPart in Stack of X
  var toAdd in Stack of X
  var index in Int
  firstPart <-- s
  toAdd <-- t
  toAdd <-- REVERSE(toAdd)
  for (index <-- 1 to SIZE(t))
  {
    firstPart <-- PUSH(PEEK(toAdd), firstPart)
    toAdd <-- POP(toAdd)
  }
  return firstPart
}
```

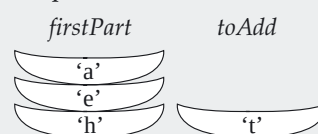
Step 2:



Step 3:



Step 4 (once):



Step 4 (twice):

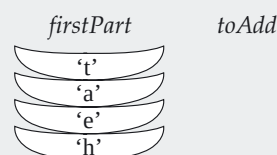


Figure 4.2

Objectives for Section 4

After studying this section you should be able to do the following.

- ❑ Interpret and use the notation associated with the data type **Stack of X**, including:
 - the type functions *SIZE*, *PEEK*, *POP* and *PUSH*;
 - use of sequence notation for stacks in traces.
- ❑ Be aware that a stack variable is created, by a **var** statement, as an empty sequence (its default state).
- ❑ Interpret a specification involving the type **Stack of X**.
- ❑ Trace code involving stack variables, and test the returned value against a given specification.
- ❑ Write implementations using **Stack of X** (and the data types **Int**, **Bool** and **Char**).

Exercises on Section 4

Exercise 4.1

Suppose that $s = [11, 2, 5, 7, 4]$. What are each of:

- (a) *SIZE*(s);
- (b) *PEEK*(s);
- (c) *POP*(s);
- (d) *POP*(*POP*(s));
- (e) *PEEK*(*POP*(s))?

Exercise 4.2

Give a specification and implementation of a function *REPSEC* that replaces the item second from the top of a stack with a new item. So, for example, *REPSEC*('Q', ['a', 'b', 'c', 'd']) = ['a', 'b', 'Q', 'd']. Include explanatory comments in your implementation.

Exercise 4.3

Give an implementation of *COUNTOF2*, specified below. You can assume that the data type **X** has a test of equality `==`.

function *COUNTOF2*(x in **X**, s in **Stack of X**) **return** in **Int**

pre true.

post The returned value gives the number of times that x appears in s . If s is empty, or if x does not appear at all in s , then the returned value is 0.

The functions *COUNTOF2* in this exercise and *COUNTOF* in Exercise 3.3 are very similar. However, the two processes have different signatures, and this is important. The signature line in *COUNTOF2* states that s is a stack. In the implementation of *COUNTOF2*, we can apply only type functions of the data type **Stack of X** to s , and we can assign s only to a local variable of data type **Stack of X**.

Giving the functions different identifiers emphasises this difference.

Summary of Unit 3

1. A data type is a data set together with type functions that are available to variables of that type. You met the primitive data types **Bool**, **Int** and **Char**. These bring together data sets and processes that you met in Units 1 and 2. In code, **var** statements are expressed in terms of data types. This lets you know which processes are available to variables.
2. When entering code into the WorkPad, only symbols on a keyboard can be used. Adjustments to the details of how the course code is expressed were introduced, to enable it to be entered into the WorkPad.

The data type of each input to a function is given in the signature line of its specification. You need to take this into account when writing implementations.

3. An array is a non-empty sequence of items whose length is fixed. The items in an array must all be of the same type. An array literal is written using the usual notation for a sequence, as in, for example, `[1,5,3,11]` or `['q', 'w', 'e', 'r', 't', 'y']`. Once an array variable has been declared, its size cannot be changed. Nor can the type of object contained in the array be changed. This leads to a need for care when making an assignment to an array variable.

The type and number of items in an array are given in the **var** statement. Immediately after a variable of type **Array of X** is created by a **var** statement, it has the size specified in the **var** statement, with all its places blank. The function *AT* cannot be applied to an array until all blanks have been filled. Code to process all the items in an array typically uses a **for** loop with a loop index corresponding to the position of each item within the array.

4. A stack is a sequence of items whose length can vary. Immediately after a variable of type **Stack of X** is created by a **var** statement, it has the default value, which is an empty sequence.

The type functions on stacks only allow you to operate at one end of the sequence (the top of the stack). You can examine the top item, delete the top item, or add a new item at the top. The restricted nature of these type functions means that implementations using the stack data type need an approach different from that for implementations using arrays.

Each of **Array of X** and **Stack of X** is a parameterised data type.

Solutions to the exercises

Solution 1.1

(a) This expression is valid (both 'b' and $CHR(70)$ are characters). The ASCII code of 'b' is 98. So the character with ASCII code 70 comes before 'b' in the ordering of characters, and so the given expression is **false**.

(b) This is valid, and has value 2. To see this, first find $DIV(37,5)$, which is 7 (since $37 = 7 \times 5 + 2$). So the given expression is equivalent to $DIV(7,3)$, which is 2 (since $7 = 2 \times 3 + 1$).

Solution 1.2

(a) $ASC(c)$ is in **Int**, as is 100. **Int** has a comparison $<$ as a type function, and so $<$ in $(ASC(c) < 100)$ can be interpreted as this type function of **Int**. This expression is valid, and returns a value in **Bool**. Similarly, in $(ASC(c) == 100)$, **Int** has a type function $==$, and so this expression is valid. It returns a value in **Bool**. The use of $\vee$ in $((ASC(c) < 100) \vee (ASC(c) == 100))$ then combines two values in **Bool**, and so is valid. This expression also returns a value in **Bool**. Now c is of type **Char** and 'a' is a literal of type **Char**. **Char** has a type function $>$, and so in $(c > 'a')$, $>$ can be interpreted as this type function of **Char**. The value returned by $(c > 'a')$ is in **Bool**. Finally, the use of $\wedge$ is then valid, since it combines two values in **Bool**. The resulting value of $anexp$ is of type **Bool**.

(b) The expression $anexp$ will be **true** if c has an ASCII code that is less than or equal to 100, and that is also greater than that of 'a'. So c must be one of the characters: 'b', 'c' or 'd'.

(c) Either of the expressions below would do:

$(b \leq c) \wedge (c \leq d), \quad (a < c) \wedge (c < e).$

Solution 1.3

There are many ways in which this condition might be expressed. Your answer only needs to be one of them! You could use either the ordering of the characters themselves (and use the comparisons $<$, $\leq$ and so on of the data type **Char**), or look at the ASCII code of the character, and use comparisons of **Int**. A character is an upper-case letter if it lies between 'A' and 'Z' (inclusive) in the ordering of **Char**. So c is an upper-case letter if $A \leq c$ and $c \leq Z$. Hence c is not an upper-case letter if the expression $NOT((c \geq A) \wedge (c \leq Z))$ is **true**. Another approach would be to say that c is not an upper-case letter if it is strictly less than 'A' or if it is strictly greater than 'Z', leading to the expression $(c < A) \vee (c > Z)$.

Alternatively, c is an upper-case letter if its ASCII code lies between 65 and 90, inclusive. So c is an upper-case letter if $ASC(c) \geq 65$ and $ASC(c) \leq 90$, and then c is not an upper-case letter if $NOT((ASC(c) \geq 65) \wedge (ASC(c) \leq 90))$ is **true**. Or you might have expressed this condition as $(ASC(c) < 65) \vee (ASC(c) > 90)$.

Any of the following correctly expresses the required condition:

$$\begin{aligned} & NOT((c \geq A) \wedge (c \leq Z)); \\ & (c < A) \vee (c > Z); \\ & NOT((ASC(c) \geq 65) \wedge (ASC(c) \leq 90)); \\ & NOT((ASC(c) \geq 65) \wedge (ASC(c) \leq 90)); \\ & (ASC(c) < 65) \vee (ASC(c) > 90); \\ & (ASC(c) \leq 64) \vee (ASC(c) \geq 91). \end{aligned}$$

Even this is not a complete list of correct answers!

Solution 2.1

(a) (i) The final state of *aChar* is 'b'. (ii) The final state of *aChar* would then be 'Q'.

(b) This is essentially a matter of packaging the code in (a). We need a precondition in the specification, to ensure that the final call of *CHR* is valid.

function *NEXTCH*(*c* in Char) **return** in Char

pre The ASCII code of *c* is less than 126.

post The returned value is the character whose ASCII code is one more than that of *c*.

```
function NEXTCH(c)
{
  var aChar in Char
  var num in Int
  aChar <-- c
  num <-- ASC(aChar)
  num <-- num + 1
  aChar <-- CHR(num)
  return aChar
}
```

The two statements
 $aChar \leftarrow c$ and
 $num \leftarrow ASC(aChar)$ could be
 replaced by the single
 statement $num \leftarrow ASC(c)$.

(c) The function *NEXTCH* does what we want, unless the input character is 'z'. In that case we want to return 'a'.

```
function NEXT(lower)
{
  var aChar in Char
  if (lower == 'z') then
    {aChar <-- 'a'}
  else
    {aChar <-- NEXTCH(lower)}
  return aChar
}
```

Solution 2.2

(a) A specification and implementation are given below.

function *HASAFACT*(*m* in Int, *n* in Int) **return** in Bool

pre $m > 1$ and $n > 1$.

post The returned value is **true** if *n* is a factor of *m* and is **false** otherwise.

```
function HASAFACT(m,n)
{
  var aBool in Bool
  if (MOD(m,n) == 0) then
    {aBool <-- true}
  else
    {aBool <-- false}
  return aBool
}
```

The conditional statement here could be replaced by the single assignment shown below.

```
aBool <-- (MOD(m,n) == 0)
```

(b) A specification and implementation are given below. In the implementation, you can use *index* in the **for** loop to run through the integer values that you need to check as possible factors of p .

function *ISPRIME*(p in Int) **return** in Bool

pre $p > 1$.

post The returned value is **true** if p is prime and is **false** otherwise.

```
function ISPRIME(p)
{
  var aBool in Bool
  var index in Int
  aBool <-- true
  for (index <-- 2 to p-1)
  {
    if (HASAFACT(p,index)) then {aBool <-- false}
  }
  return aBool
}
```

Solution 3.1

(a) The item at position 4 in the array *arr* is the integer 24. So $AT(4, arr) = 24$.

(b) Remember that in *PUT*, the first input is the position of the item to be changed and the third input is the new item. If we change the item in position 5 in *arr* to 1, we get $PUT(5, arr, 1) = [21, 22, 23, 24, 1, 26]$. (c) If we change the item in position 1 in *arr* to 5, we get $PUT(1, arr, 5) = [5, 22, 23, 24, 25, 26]$.

Solution 3.2

(a) $AT(2, arr)$.

(b) (i) $AT(2, [5, 7, 22, -13, 0, 2]) = 7$. (ii) $AT(2, ['a', 'r', 'r', 'a', 'y']) = 'r'$.

(c) We give a specification and implementation below.

function *ARRSEC*(*arr* in Array of X) **return** in X

pre The array *arr* is full and $SIZE(arr) \geq 2$.

post The returned value is the second item in *arr*.

```
function ARRSEC(arr)
{
  return AT(2, arr)
}
```

Solution 3.3

(a) An implementation is given below.

```
function COUNTOF(item,arr)
{
  var index in Int
  var count in Int
  count <-- 0
  for (index <-- 1 to SIZE(arr))
  {
    if (AT(index,arr) == item) then
      {count <-- count + 1}
  }
  return count
}
```

(b) With inputs 'a' and ['a', 'a', 'r', 'd', 'v', 'a'], the postcondition states that the returned value should be 3 (the number of times 'a' appears in ['a', 'a', 'r', 'd', 'v', 'a']).

A trace with these inputs is given below.

Trace with inputs $item = 'a'$ and $arr = ['a', 'a', 'r', 'd', 'v', 'a']$

Comment	$index$	$AT(index, arr) == item$	$count$
$count \leftarrow 0$	-	-	0
loop	1	'a' == 'a' is true	1
	2	'a' == 'a' is true	2
	3	'r' == 'a' is false	2
	4	'd' == 'a' is false	2
	5	'v' == 'a' is false	2
loop end	6	'a' == 'a' is true	3

Since $SIZE(['a', 'a', 'r', 'd', 'v', 'a']) = 6$, the body of the **for** loop is executed 6 times. The returned value is 3, which is correct.

Solution 4.1

- (a) There are five items in [11, 2, 5, 7, 4]; so $SIZE(s) = 5$.
- (b) The stack is written as a sequence with the top item on the right.
So $PEEK(s) = 4$.
- (c) To form $POP(s)$, delete the top item from s . So $POP(s) = [11, 2, 5, 7]$.
- (d) To form $POP(POP(s))$, delete the top item from $POP(s)$.
So $POP(POP(s)) = [11, 2, 5]$.
- (e) $PEEK(POP(s))$ is the top (right-most) item in $POP(s)$.
So $PEEK(POP(s)) = 7$.

Solution 4.2

Although the given example involves characters, the description of *REPSEC* can apply to stacks whose items may be of any type, say **X**. This function has two inputs, one (the replacement item) from **X** and one from **Stack of X**, and returns a stack. This must be shown in the signature line. There will need to be at least two items in the input stack (so that there is a second item to replace). This must be shown in the precondition.

The function *REPSEC* can be implemented by the following sequence of actions.

1. Save the top item in the stack in a local variable (using *PEEK*).
2. Initialise a variable stack, *newStack*, as the input stack.
3. Delete the top two items from *newStack* (using *POP*).
4. Put the replacement item on top of *newStack* (using *PUSH*).
5. Replace the saved top item (using *PUSH*).

function *REPSEC*(*x* in **X**, *s* in **Stack of X**) **return** in **Stack of X**

pre The stack *s* contains at least two items.

post The returned stack is formed by replacing the item second from the top in *s* with *x*. So, for example, *REPSEC*('Q', ['a', 'b', 'c', 'd']) = ['a', 'b', 'Q', 'd'].

```
function REPSEC(x,s)
{
  var newStack in Stack of X
  var anX in X
  newStack <-- s
  anX <-- PEEK(s)
  /* The top item in s is stored in anX. */
  newStack <-- POP(newStack)
  /* Deletes the top item in s from newStack. */
  newStack <-- POP(newStack)
  /* Deletes the item second from top of s from newStack. */
  newStack <-- PUSH(x, newStack)
  /* Puts the new item (x) onto newStack. */
  newStack <-- PUSH(anX, newStack)
  /* Puts the top item from s on top of newStack. */
  return newStack
}
```

Solution 4.3

An implementation is given below.

```
function COUNTOF2(x,s)
{
  var count in Int
  var stackLeft in Stack of X
  var index in Int
  count <-- 0
  stackLeft <-- s
  for (index <-- 1 to SIZE(s))
  {
    if (PEEK(stackLeft) == x) then
      {count <-- count + 1}
    stackLeft <-- POP(stackLeft)
  }
  /* count gives the number of copies of x that
  have been deleted from stackLeft */
  return count
}
```


Index

- array, 24
 - full, 25
- binary notation, 21
- blanks in an array, 25
- case sensitive, 14
- concatenation, 40
- data set, 5
- data type, 5
 - Array of X**, 26
 - Bool**, 6
 - Char**, 11
 - Float**, 10
 - Int**, 7, 8
 - Stack of X**, 35
- decimal notation, 21
- default, 34
- floating point, 10
- formal generic parameter, 27
- generic type, 27
- integer
 - factor of a, 23
 - prime, 23
- literal, 14
- overloading, 11
- parameterised type, 27
- primitive data types, 4
- real numbers, 10
- stack, 34
- strongly typed, 14
- syntax, 17
- type function, 5
 - identifier of a, 5
- variable
 - type, 5
- WorkPad, 2